

```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My Drive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/My Drive/$FOLDERNAME
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force\_remount=True).

```
/content/drive/My Drive/cs231n/assignments/assignment1/cs231n/datasets
/content/drive/My Drive/cs231n/assignments/assignment1
```

## ✓ Fully-Connected Neural Nets

In this exercise we will implement fully-connected networks using a modular approach. For each layer we will implement a `forward` and a `backward` function. The `forward` function will receive inputs, weights, and other parameters and will return both an output and a `cache` object storing data needed for the backward pass, like this:

```
def layer_forward(x, w):
    """ Receive inputs x and weights w """
    # Do some computations ...
    z = # ... some intermediate value
    # Do some more computations ...
    out = # the output

    cache = (x, w, z, out) # Values we need to compute gradients

    return out, cache
```

The backward pass will receive upstream derivatives and the `cache` object, and will return gradients with respect to the inputs and weights, like this:

```
def layer_backward(dout, cache):
    """
    Receive dout (derivative of loss with respect to outputs) and cache,
    and compute derivative with respect to inputs.
    """
    # Unpack cache values
    x, w, z, out = cache

    # Use values in cache to compute derivatives
    dx = # Derivative of loss with respect to x
    dw = # Derivative of loss with respect to w

    return dx, dw
```

After implementing a bunch of layers this way, we will be able to easily combine them to build classifiers with different architectures.

```
# As usual, a bit of setup
from __future__ import print_function
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient, eval_numerical_gradient_array
from cs231n.solver import Solver
```

```
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """ returns relative error """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

```
# Load the (preprocessed) CIFAR10 data.
```

```
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print('%s: ' % k, v.shape)
```

```
('X_train: ', (49000, 3, 32, 32))
('y_train: ', (49000,))
('X_val: ', (1000, 3, 32, 32))
('y_val: ', (1000,))
('X_test: ', (1000, 3, 32, 32))
('y_test: ', (1000,))
```

## ✓ Affine layer: forward

Open the file `cs231n/layers.py` and implement the `affine_forward` function.

Once you are done you can test your implementaion by running the following:

```
# Test the affine_forward function

num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_shape)
w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape), output_dim)
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,  3.77273342]])

# Compare your output with ours. The error should be around e-9 or less.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing affine_forward function:
difference: 9.769849468192957e-10
```

## ✓ Affine layer: backward

Now implement the `affine_backward` function and test your implementation using numeric gradient checking.

```
# Test the affine_backward function
np.random.seed(231)
x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[0], b, dout)

_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

# The error should be around e-10 or less
print('Testing affine_backward function:')
```

```
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_backward function:
dx error: 5.399100368651805e-11
dw error: 9.904211865398145e-11
db error: 2.4122867568119087e-11
```

## ✓ ReLU activation: forward

Implement the forward pass for the ReLU activation function in the `relu_forward` function and test your implementation using the following:

```
# Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.,          ],
                        [ 0.,          0.,          0.,          0.04545455, 0.13636364, ],
                        [ 0.22727273, 0.31818182, 0.40909091, 0.5,          ]])

# Compare your output with ours. The error should be on the order of e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))

Testing relu_forward function:
difference: 4.999999798022158e-08
```

## ✓ ReLU activation: backward

Now implement the backward pass for the ReLU activation function in the `relu_backward` function and test your implementation using numeric gradient checking:

```
np.random.seed(231)
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be on the order of e-12
print('Testing relu_backward function:')
print('dx error: ', rel_error(dx_num, dx))

Testing relu_backward function:
dx error: 3.2756349136310288e-12
```

### Inline Question 1:

We've only asked you to implement ReLU, but there are a number of different activation functions that one could use in neural networks, each with its pros and cons. In particular, an issue commonly seen with activation functions is getting zero (or close to zero) gradient flow during backpropagation. Which of the following activation functions have this problem? If you consider these functions in the one dimensional case, what types of input would lead to this behaviour?

1. Sigmoid
2. ReLU
3. Leaky ReLU

*colorblue Your Answer :*

#### 1. Sigmoid (S型激活函数)

问题：有。

**导致该行为的输入类型：**当输入  $x$  变得非常大（正无穷大）或非常小（负无穷大）时。在这种情况下，Sigmoid 函数会饱和，其导数会变得非常接近于零。这意味着流经该神经元的梯度会非常小，导致其权重更新微乎其微，从而引发梯度消失问题。

#### 2. ReLU (修正线性单元)

问题：有（被称为“垂死ReLU”问题）。

**导致该行为的输入类型：**当输入  $x$  小于或等于零时。对于  $x \leq 0$ ，ReLU 的输出为零，其导数也为零。这意味着如果一个 ReLU 神经元在训练过程中始终接收到负值或零输入，它将永远不会被激活，并且其权重将不会通过反向传播进行更新，从而导致该神经元“死亡”。

### 3. Leaky ReLU (渗漏修正线性单元)

**问题：**基本上没有。

**导致该行为的输入类型：**Leaky ReLU 的设计目的就是为了解决 ReLU 的“垂死”问题。当输入  $x$  小于零时，Leaky ReLU 的导数是一个小的正数（例如 0.01），而不是零。因此，即使对于负输入，梯度也仍然会流动，从而有效缓解了梯度消失的问题。它通常不会在任何输入类型下出现梯度接近于零的问题，除非其“渗漏”参数本身设置为零（此时就退化为普通 ReLU）。

## ✧ "Sandwich" layers

There are some common patterns of layers that are frequently used in neural nets. For example, affine layers are frequently followed by a ReLU nonlinearity. To make these common patterns easy, we define several convenience layers in the file `cs231n/layer_utils.py`.

For now take a look at the `affine_relu_forward` and `affine_relu_backward` functions, and run the following to numerically gradient check the backward pass:

```
from cs231n.layer_utils import affine_relu_forward, affine_relu_backward
np.random.seed(231)
x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w, b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w, b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w, b)[0], b, dout)

# Relative error should be around e-10 or less
print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))

Testing affine_relu_forward and affine_relu_backward:
dx error:  2.299579177309368e-11
dw error:  8.162011105764925e-11
db error:  7.826724021458994e-12
```

## ✧ Loss layers: Softmax

Now implement the loss and gradient for softmax in the `softmax_loss` function in `cs231n/layers.py`. These should be similar to what you implemented in `cs231n/classifiers/softmax.py`. Other loss functions (e.g. `svm_loss`) can also be implemented in a modular way, however, it is not required for this assignment.

You can make sure that the implementations are correct by running the following:

```
np.random.seed(231)
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)

dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x, verbose=False)
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be close to 2.3 and dx error should be around e-8
print('\nTesting softmax_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))

Testing softmax_loss:
loss:  2.3025458445007376
dx error:  8.234144091578429e-09
```

## ✧ Two-layer network

Open the file `cs231n/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class. Read through it to make sure you understand the API. You can run the cell below to test your implementation.

```
np.random.seed(231)
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-3
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C, weight_scale=std)

print('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
    [[11.53165108, 12.2917344, 13.05181771, 13.81190102, 14.57198434, 15.33206765, 16.09215096],
     [12.05769098, 12.74614105, 13.43459113, 14.1230412, 14.81149128, 15.49994135, 16.18839143],
     [12.58373087, 13.20054771, 13.81736455, 14.43418138, 15.05099822, 15.66781506, 16.2846319 ]])
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss'

model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization loss'

# Errors should be around e-7 or less
for reg in [0.0, 0.7]:
    print('Running numeric gradient check with reg = ', reg)
    model.reg = reg
    loss, grads = model.loss(X, y)

    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name], verbose=False)
        print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))
```

```
Testing initialization ...
Testing test-time forward pass ...
Testing training loss (no regularization)
Running numeric gradient check with reg = 0.0
W1 relative error: 1.83e-08
W2 relative error: 3.20e-10
b1 relative error: 9.83e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg = 0.7
W1 relative error: 2.53e-07
W2 relative error: 2.85e-08
b1 relative error: 1.56e-08
b2 relative error: 9.09e-10
```

## ✓ Solver

Open the file `cs231n/solver.py` and read through it to familiarize yourself with the API. After doing so, use a `Solver` instance to train a `TwoLayerNet` that achieves about `36%` accuracy on the validation set.

```
input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
```

```

model = TwoLayerNet(input_size, hidden_size, num_classes)

#####
# TODO: Use a Solver instance to train a TwoLayerNet that achieves about 36% #
# accuracy on the validation set.                                           #
#####
solver = Solver(model, data,
                 update_rule='sgd',
                 optim_config={
                     'learning_rate': 1e-4,
                 },
                 lr_decay=0.95,
                 num_epochs=5, batch_size=200,
                 print_every=100,
                 verbose=True)

solver.train()
#####
#                                     END OF YOUR CODE
#####

```

```

(Iteration 1 / 1225) loss: 2.302913
(Epoch 0 / 5) train acc: 0.076000; val_acc: 0.073000
(Iteration 101 / 1225) loss: 2.249682
(Iteration 201 / 1225) loss: 2.145013
(Epoch 1 / 5) train acc: 0.247000; val_acc: 0.250000
(Iteration 301 / 1225) loss: 2.074358
(Iteration 401 / 1225) loss: 1.981304
(Epoch 2 / 5) train acc: 0.280000; val_acc: 0.299000
(Iteration 501 / 1225) loss: 1.981099
(Iteration 601 / 1225) loss: 1.849081
(Iteration 701 / 1225) loss: 1.909062
(Epoch 3 / 5) train acc: 0.308000; val_acc: 0.329000
(Iteration 801 / 1225) loss: 1.800961
(Iteration 901 / 1225) loss: 1.837831
(Epoch 4 / 5) train acc: 0.347000; val_acc: 0.355000
(Iteration 1001 / 1225) loss: 1.866895
(Iteration 1101 / 1225) loss: 1.700069
(Iteration 1201 / 1225) loss: 1.837567
(Epoch 5 / 5) train acc: 0.375000; val_acc: 0.371000

```

## ▼ Debug the training

With the default parameters we provided above, you should get a validation accuracy of about 0.36 on the validation set. This isn't very good.

One strategy for getting insight into what's wrong is to plot the loss function and the accuracies on the training and validation sets during optimization.

Another strategy is to visualize the weights that were learned in the first layer of the network. In most neural networks trained on visual data, the first layer weights typically show some visible structure when visualized.

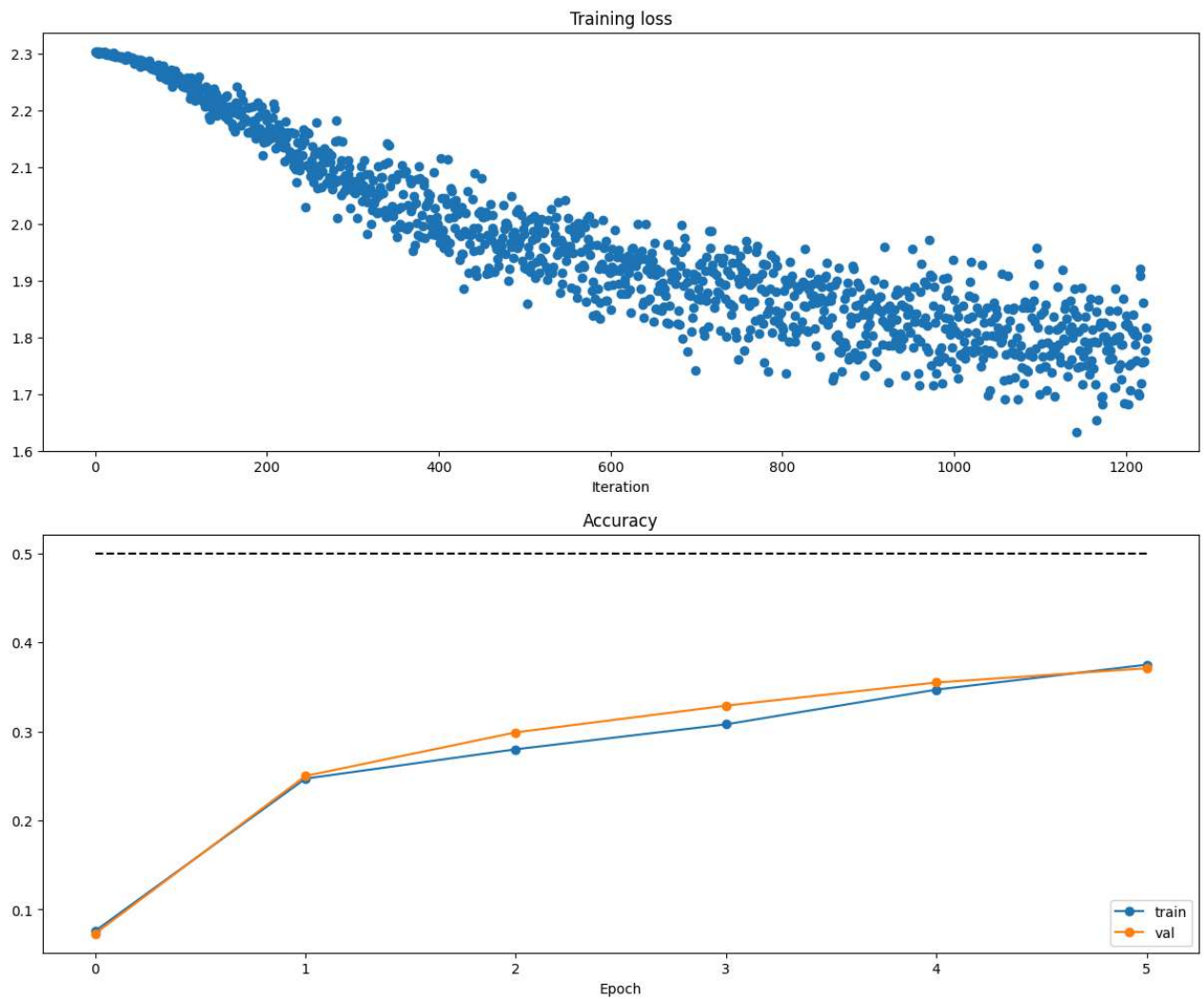
```

# Run this cell to visualize training loss and train / val accuracy

plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()

```



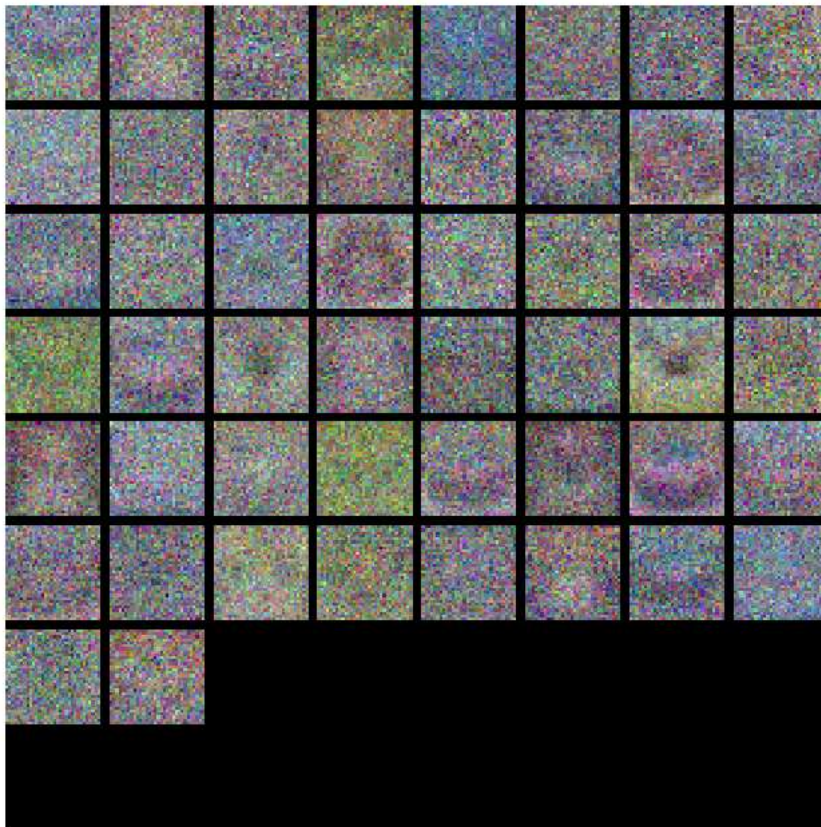
```
from cs231n.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(3, 32, 32, -1).transpose(3, 1, 2, 0)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(model)
```





## ✓ Tune your hyperparameters

**What's wrong?** Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

**Tuning.** Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, number of training epochs, and regularization strength. You might also consider tuning the learning rate decay, but you should be able to get good performance using the default value.

**Approximate results.** You should be able to achieve a classification accuracy of greater than 48% on the validation set. Our best network gets over 52% on the validation set.

**Experiment:** Your goal in this exercise is to get as good of a result on CIFAR-10 as you can (52% could serve as a reference), with a fully-connected Neural Network. Feel free to implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

```
best_model = None

#####
# TODO: Tune hyperparameters using the validation set. Store your best trained #
# model in best_model.
#
# To help debug your network, it may help to use visualizations similar to the #
# ones we used above; these visualizations will have significant qualitative #
# differences from the ones we saw above for the poorly tuned network.
#
# Tweaking hyperparameters by hand can be fun, but you might find it useful to #
# write code to sweep through possible combinations of hyperparameters
# automatically like we did on the previous exercises.
#####
best_val = -1

# 设置需要调优的超参数范围
# 增加模型容量 (hidden_sizes) 和尝试更快的学习率 (learning_rates)
hidden_sizes = [100, 200]
learning_rates = [5e-4, 1e-3, 2e-3]
reg_strengths = [0.1, 0.25, 0.5]
```



```

input_size = 32 * 32 * 3
num_classes = 10

for hs in hidden_sizes:
    for lr in learning_rates:
        for reg in reg_strengths:
            # 初始化双层神经网络
            model = TwoLayerNet(input_dim=input_size,
                                hidden_dim=hs,
                                num_classes=num_classes,
                                weight_scale=1e-3,
                                reg=reg)

            # 使用 Solver 进行训练
            solver = Solver(model, data,
                             update_rule='sgd',
                             optim_config={'learning_rate': lr},
                             lr_decay=0.95,
                             num_epochs=10,
                             batch_size=200,
                             print_every=500,
                             verbose=False)

            solver.train()

            # 检查当前模型是否为目前最好的
            if solver.best_val_acc > best_val:
                best_val = solver.best_val_acc
                best_model = model

            print('隐层大小: %d, 学习率: %e, 正则化: %e, 验证集准确率: %f' % (hs, lr, reg, solver.best_val_acc))

print('交叉验证完成。最高验证集准确率: %f' % best_val)
#####
#                                     END OF YOUR CODE
#####

```

```

隐层大小: 100, 学习率: 5.000000e-04, 正则化: 1.000000e-01, 验证集准确率: 0.518000
隐层大小: 100, 学习率: 5.000000e-04, 正则化: 2.500000e-01, 验证集准确率: 0.516000
隐层大小: 100, 学习率: 5.000000e-04, 正则化: 5.000000e-01, 验证集准确率: 0.506000
隐层大小: 100, 学习率: 1.000000e-03, 正则化: 1.000000e-01, 验证集准确率: 0.519000
隐层大小: 100, 学习率: 1.000000e-03, 正则化: 2.500000e-01, 验证集准确率: 0.520000
隐层大小: 100, 学习率: 1.000000e-03, 正则化: 5.000000e-01, 验证集准确率: 0.517000
隐层大小: 100, 学习率: 2.000000e-03, 正则化: 1.000000e-01, 验证集准确率: 0.507000
隐层大小: 100, 学习率: 2.000000e-03, 正则化: 2.500000e-01, 验证集准确率: 0.492000
隐层大小: 100, 学习率: 2.000000e-03, 正则化: 5.000000e-01, 验证集准确率: 0.506000
隐层大小: 200, 学习率: 5.000000e-04, 正则化: 1.000000e-01, 验证集准确率: 0.516000
隐层大小: 200, 学习率: 5.000000e-04, 正则化: 2.500000e-01, 验证集准确率: 0.523000
隐层大小: 200, 学习率: 5.000000e-04, 正则化: 5.000000e-01, 验证集准确率: 0.508000
隐层大小: 200, 学习率: 1.000000e-03, 正则化: 1.000000e-01, 验证集准确率: 0.530000
隐层大小: 200, 学习率: 1.000000e-03, 正则化: 2.500000e-01, 验证集准确率: 0.526000
隐层大小: 200, 学习率: 1.000000e-03, 正则化: 5.000000e-01, 验证集准确率: 0.541000
隐层大小: 200, 学习率: 2.000000e-03, 正则化: 1.000000e-01, 验证集准确率: 0.522000
隐层大小: 200, 学习率: 2.000000e-03, 正则化: 2.500000e-01, 验证集准确率: 0.515000
隐层大小: 200, 学习率: 2.000000e-03, 正则化: 5.000000e-01, 验证集准确率: 0.508000
交叉验证完成。最高验证集准确率: 0.541000

```

## ✓ Test your model!

Run your best model on the validation and test sets. You should achieve above 48% accuracy on the validation set and the test set.

```

y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())

```

```
Validation set accuracy: 0.541
```

```

y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())

```

```
Test set accuracy: 0.533
```

```

# Save best model
best_model.save("best_two_layer_net.npy")

```

```
best_two_layer_net.npy saved.
```

## ✓ Inline Question 2:

Now that you have trained a Neural Network classifier, you may find that your testing accuracy is much lower than the training accuracy. In what ways can we decrease this gap? Select all that apply.

1. Train on a larger dataset.
2. Add more hidden units.
3. Increase the regularization strength.
4. None of the above.

*Your Answer : 1, 3*

*Your Explanation :*

当测试准确率远低于训练准确率时，这通常表明模型出现了过拟合（Overfitting）。过拟合意味着模型在训练数据上学习得太好，包括了噪声和特定样本的细节，导致其泛化能力差，在未见过的数据（测试集）上表现不佳。

针对过拟合问题，我们可以采取以下策略来减小训练与测试准确率之间的差距：

1. **训练更大的数据集：** 增加训练数据的多样性和数量可以帮助模型学习到更普遍、更鲁棒的模式，而不是仅仅记住训练集中的特定样本。这使得模型更难以过度拟合单一的数据点或噪声，从而提高了其泛化能力。
2. **增加正则化强度：** 正则化是一种惩罚模型复杂度的技术。例如，L1或L2正则化通过在损失函数中添加权重大小的惩罚项，鼓励模型使用更小的权重，从而降低模型的复杂度，减少对训练数据的过度依赖。Dropout也是一种有效的正则化方法，通过随机丢弃神经元来防止网络过拟合。增加正则化强度会迫使模型学习更简单的表示，从而改善泛化性能。

**不正确的选项解释：**